Haskell

Часть 2. Типы и организация кода

Андреев Александр
Крапивина Ульяна
Савосина Дарья

Частичные функции и ошибки

01 Тип результата не гарантирует успешного вычисления.

02 head [] и error вызывают исключение при вычислении.

03 Ожидаемую неудачу удобно выразить обычным значением.

```
head [1,2,3]  -- 1
head []       -- exception

error "broken invariant" :: Int
undefined :: Int
```

Чистота функции не означает отсутствие исключений или бесконечного вычисления.

Maybe: значение может отсутствовать

01 Maybe a содержит Nothing либо Just a.

02 Пустой список становится обычным результатом Nothing.

03 Обрабатываем результат сопоставлением с образцом.

```
safeHead :: [a] -> Maybe a
safeHead []    = Nothing
safeHead (x:_) = Just x

safeHead [3,4]  -- Just 3
safeHead []     -- Nothing
```

Maybe выражает отсутствие, но не хранит объяснение причины.

Either: результат с причиной ошибки

01 Left e хранит ошибку, Right a хранит результат.

02 Отдельный тип перечисляет допустимые причины.

03 Соглашение Left / Right помогает читать интерфейс.

```
data DivError = ZeroDivisor
  deriving (Eq, Show)

safeDiv :: Integer -> Integer
        -> Either DivError Integer
safeDiv _ 0 = Left ZeroDivisor
safeDiv x y = Right (x `div` y)
```

safeDiv 10 2 = Right 5. safeDiv 10 0 = Left ZeroDivisor.

Обработка Maybe и Either

01 case позволяет явно разобрать каждую альтернативу.

02 maybe задаёт значение по умолчанию.

03 either принимает отдельный обработчик для Left и Right.

```
case safeHead [3,4] of
  Nothing -> "empty"
  Just x  -> show x

maybe 0 (+1) (Just 5)  -- 6
maybe 0 (+1) Nothing   -- 0

either show show (safeDiv 10 0)
-- "ZeroDivisor"
```

Выбор значения по умолчанию должен соответствовать смыслу задачи.

Исключения и перехват в IO

01 Исключения возникают и в чистом коде, и в IO.

02 try возвращает исключение как Left.

03 Тип e определяет, какие исключения перехватываются.

```
import Control.Exception
  (ArithException, evaluate, try)

main :: IO ()
main = do
  r <- try (evaluate (10 `div` 0))
        :: IO (Either ArithException Int)
  print r
-- Left divide by zero
```

Общий тип try: Exception e => IO a -> IO (Either e a).

Ленивость и границы обработчика

01 Исключение возникает при востребовании выражения.

02 evaluate вычисляет до слабой головной формы, WHNF.

03 Вложенные элементы списка могут оставаться невычисленными.

```
evaluate [1, error "later"]
-- the list constructor is available

evaluate (length [1, error "later"])
-- 2: elements are not evaluated

evaluate (sum [1, error "later"])
-- exception: later
```

Обработчик должен охватывать вычисление, в котором действительно проявляется ошибка.

assert, trace и предупреждения GHC

01 assert проверяет инвариант во время вычисления.

02 При -fignore-asserts проверки могут отсутствовать.

03 trace помогает увидеть востребование выражения.

04 -Wall и -Wincomplete-patterns находят часть проблем заранее.

```
import Control.Exception (assert)
import Debug.Trace (trace)

checked :: Int -> Int
checked n = assert (n >= 0) (n + 1)

doubled :: Int -> Int
doubled n = trace "doubling" (n * 2)
```

assert не заменяет проверку пользовательского ввода. trace не заменяет обычный IO.

Модуль как единица организации

01 Модуль объединяет связанные определения.

02 Шапка задаёт имя и публичные определения.

03 Внутренние помощники можно скрыть.

```
-- Lecture/Safe.hs
module Lecture.Safe (safeHead) where

safeHead :: [a] -> Maybe a
safeHead []    = Nothing
safeHead (x:_) = Just x

-- Main file
import Lecture.Safe (safeHead)
```

В обычном проекте GHC модулю Lecture.Safe соответствует файл Lecture/Safe.hs.

Пространства имён и импорт

01 Иерархическое имя организует модули.

02 qualified добавляет префикс к использованию имён.

03 Явный список импорта уменьшает неоднозначность.

```
import Data.List (sort)
import qualified Data.Map.Strict as M
import qualified Data.Set as S

M.lookup "a" (M.fromList [("a", 3)])
-- Just 3

S.member 3 (S.fromList [1,3])
-- True
```

Список импорта и hiding ограничивают доступные имена. qualified устраняет конфликты.

Экспорт и защита инвариантов

01 Можно экспортировать тип без его конструктора.

02 Создание значения проходит через проверяющую функцию.

03 T(..) открывает все конструкторы типа T.

```
module Lecture.Money
  (Money, mkMoney, amount) where

newtype Money = Money Int

mkMoney :: Int -> Maybe Money
mkMoney n
  | n < 0     = Nothing
  | otherwise = Just (Money n)
```

Клиент видит Money, но не может напрямую написать Money (-1).

Пакет, библиотека и исполняемая программа

01 Пакет содержит компоненты, модули и метаданные.

02 base поставляет основную библиотеку GHC.

03 containers содержит Map, Set и Seq, array содержит Array.

04 Cabal описывает зависимости и собирает компоненты.

```
cabal-version: 2.4
name: lecture02
version: 0.1.0.0

executable modules-demo
  main-is: p05_modules.hs
  other-modules: Lecture.Safe, Lecture.Money
  build-depends: base >=4.15 && <5
  default-language: Haskell2010
```

cabal build и cabal run собирают и запускают компонент. Hackage распространяет пакеты.

Параметризованный тип

01 a обозначает переменную типа.

02 Box Int и Box String используют одно объявление.

03 Конструктор Box упаковывает значение, образец извлекает его.

```
data Box a = Box a
  deriving (Eq, Show)

unbox :: Box a -> a
unbox (Box x) = x

number :: Box Int
number = Box 7
```

Box слева от знака = является конструктором типа, справа конструктором данных.

Параметрический полиморфизм

01 Одна реализация работает для разных типов.

02 Без ограничений функция не может произвольно исследовать a.

03 Вывод типов часто избавляет от повторных аннотаций.

```
keep :: a -> a
keep x = x

keep (7 :: Int)   -- 7
keep "hello"      -- "hello"

mapBox :: (a -> b) -> Box a -> Box b
mapBox f (Box x) = Box (f x)
```

Параметр типа конкретизируется при использовании. Это не приведение значения.

Классы типов задают доступные операции

01 Eq даёт сравнение на равенство, Ord задаёт порядок.

02 Show преобразует значение в строковое представление.

03 Ограничение => указывает требуемое поведение.

```
same :: Eq a => a -> a -> Bool
same x y = x == y

render :: Show a => a -> String
render x = show x

maximumMaybe :: Ord a => [a] -> Maybe a
maximumMaybe [] = Nothing
maximumMaybe xs = Just (maximum xs)
```

Eq a => не обозначает наследование объекта. Это ограничение на тип a.

instance и deriving

01 instance реализует методы класса для конкретного типа.

02 deriving автоматически строит некоторые экземпляры.

03 Порядок конструкторов влияет на производный Ord.

```
data Status = New | Done
  deriving (Eq, Ord, Show)

class Describable a where
  describe :: a -> String

instance Describable Status where
  describe New  = "new"
  describe Done = "done"
```

New < Done = True. describe Done = "done".

Функциональный, объектный и протокольный подходы

01 Основная модель Haskell: значения и преобразующие их функции.

02 Модули скрывают представление, ADT описывают варианты данных.

03 Класс типов похож на контракт поведения для нескольких типов.

04 Классы типов не создают объектную иерархию подтипов.

```
label :: Describable a => a -> String
label x = "status: " ++ describe x

label Done  -- "status: done"

-- functions are ordinary values
map describe [New, Done]
-- ["new", "done"]
```

Привычные задачи инкапсуляции и повторного использования решаются модулями и полиморфизмом.

Списки: последовательный обход

01 Список хранит голову и хвост.

02 Добавление в начало занимает O(1).

03 Индексирование и проход по длине требуют O(n).

```
xs = [1,2,3] :: [Int]
ys = 0 : xs

map (*2) xs       -- [2,4,6]
filter odd xs     -- [1,3]
foldr (+) 0 xs    -- 6
xs !! 2          -- 3
```

xs !! i и head частичны. Доступ по индексу не является сильной стороной списка.

Array: неизменяемый массив

01 Индексы имеют фиксированный диапазон.

02 Для обычного целочисленного индекса доступ занимает O(1).

03 Обновление // возвращает новый массив.

04 Изменяемые массивы доступны отдельно через IO и ST.

```
import Data.Array (Array, (!), (//), listArray)

a :: Array Int String
a = listArray (1,3) ["one","two","three"]

a ! 2            -- "two"
b = a // [(2,"TWO")]
b ! 2            -- "TWO"
a ! 2            -- "two"
```

Операция ! требует допустимого индекса. Array находится в пакете array.

Set: уникальные элементы

01 Множество удаляет повторения.

02 Ord a позволяет организовать поиск по упорядоченному дереву.

03 member и insert имеют сложность O(log n).

```
import qualified Data.Set as S

s = S.fromList [3,1,3,2] :: S.Set Int
S.toAscList s           -- [1,2,3]
S.member 2 s            -- True
S.toAscList (S.insert 4 s)
-- [1,2,3,4]
S.toAscList s           -- [1,2,3]
```

Set хранит уникальные элементы в порядке Ord, а не в порядке вставки.

Map: поиск значения по ключу

01 Ключи уникальны и упорядочены через Ord.

02 lookup возвращает Maybe, потому что ключ может отсутствовать.

03 Strict-вариант форсирует значения до WHNF при сохранении.

```
import qualified Data.Map.Strict as M

m = M.fromList [("a",1),("b",2)]
    :: M.Map String Int

M.lookup "a" m  -- Just 1
M.lookup "z" m  -- Nothing
M.lookup "a" (M.insert "a" 7 m)
-- Just 7
```

Поиск и вставка: O(log n) сравнений ключей. Стоимость сравнения ключа тоже имеет значение.

Seq: работа с обоими концами

01 Seq хранит конечную последовательность.

02 <| и |> добавляют элемент с начала и с конца.

03 lookup безопасно возвращает элемент по индексу.

```
import qualified Data.Sequence as Q
import Data.Foldable (toList)

s = Q.fromList [1,2,3] :: Q.Seq Int
t = 0 Q.<| (s Q.|> 4)

toList t       -- [0,1,2,3,4]
Q.lookup 2 t   -- Just 2
Q.lookup 9 t   -- Nothing
```

Добавление с концов амортизированно O(1). Индексный доступ логарифмический.

Выбор коллекции

Коллекция	Типичный сценарий	Основная стоимость
[a]	Последовательный обход	(:) O(1), индекс O(n)
Array Int a	Фиксированные индексы	(!) O(1), (//) обычно O(n)
Set a	Уникальность	member / insert O(log n)
Map k v	Поиск по ключу	lookup / insert O(log n)
Seq a	Очередь, дек	Концы O(1) аморт., индекс O(log n)

n обозначает размер коллекции. Оценки не включают полное вычисление элементов.

Ошибки, модули и коллекции вместе

01 Словарь возвращает Maybe при поиске.

02 Публичный интерфейс уточняет причину через Either.

03 Такой код остаётся обычной чистой функцией.

```
data LookupError = UnknownKey String
  deriving (Eq, Show)

findValue :: String -> M.Map String Int
          -> Either LookupError Int
findValue key table =
  case M.lookup key table of
    Nothing -> Left (UnknownKey key)
    Just n  -> Right n
```

M обозначает Data.Map.Strict. Пример объединяет темы без файлового или сетевого IO.

Итоги второй части

01 Maybe и Either делают ожидаемую неудачу частью интерфейса.

02 Исключения перехватываются в IO с учётом ленивости.

03 Модули и пакеты разделяют публичный API и реализацию.

04 Полиморфизм и классы типов позволяют обобщать код.

05 Коллекцию выбирают по нужным операциям.

```
-- Next part
-- console and files
-- concurrency and parallelism
-- do, monads, laziness
-- GHC runtime and memory
```

В третьей части рассмотрим выполнение программы и взаимодействие с внешним миром.